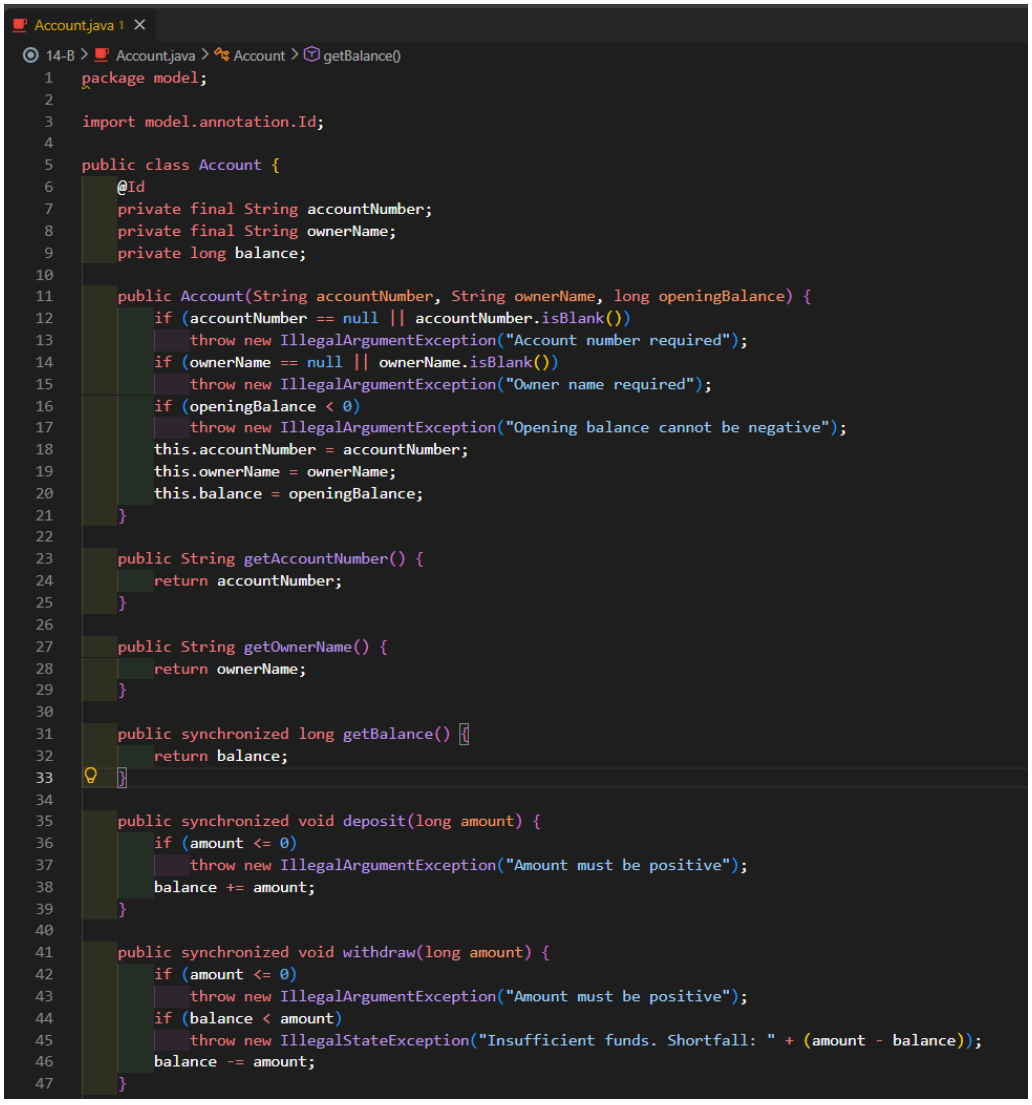
PRACTICAL 14-B

Object Oriented Programming

AIM:

To complete the MiniBank capstone by integrating BankService with Repository<Account>, validation and exception handling; connect the menu to real banking operations, load and save account data, process transaction batches safely, generate reports, write JUnit tests, and package the final runnable JAR.

CODE SCREENSHOT:



Code Screenshot 1

Account.java 1 AccountFileStore.java 1

14-B > AccountFileStore.java > AccountFileStore > save(List<Account>)

```java
package service;

import model.Account;
import java.io.*;
import java.nio.file.*;
import java.util.List;

public class AccountFileStore {
    private final Path file;

    public AccountFileStore(Path file) {
        this.file = file;
    }

    public void save(List<Account> accounts) throws IOException {
        Files.createDirectories(file.toAbsolutePath().getParent());
        try (BufferedWriter writer = Files.newBufferedWriter(file)) {
            for (Account a : accounts) {
                writer.write(a.getAccountNumber() + "|" + a.getOwnerName() + "|" + a.getBalance());
                writer.newLine();
            }
        }
    }

    public void loadInto(BankService service) throws IOException {
        if (!Files.exists(file))
            return;
        for (String line : Files.readAllLines(file)) {
            String[] p = line.split("\\|", 3);
            if (p.length == 3 && service.getRepository().findById(p[0]) == null) {
                service.getRepository().save(new Account(p[0], p[1], Long.parseLong(p[2])));
            }
        }
    }
}

```

Code Screenshot 2

AccountNotFoundException.java 1

14-B > AccountNotFoundException.java > ...

```java
package exception;

public class AccountNotFoundException extends BankException {
    public AccountNotFoundException(String accountNumber) {
        super("Account not found: " + accountNumber);
    }
}

```

Code Screenshot 3

BankException.java 1

14-B > BankException.java > ...

```java
package exception;

public class BankException extends Exception {
    public BankException(String message) {
        super(message);
    }
}

```

Code Screenshot 4

BankService.java 1

14-B > BankService.java > BankService

```java
import model.Account;
import repository.Repository;
import exception.*;
import java.util.List;
import java.util.concurrent.atomic.AtomicInteger;

public class BankService {
    private final Repository<Account> repository;
    private final AtomicInteger nextId = new AtomicInteger(1);

    public BankService(Repository<Account> repository) {
        this.repository = repository;
    }

    public Account openAccount(String ownerName, long openingBalance) {
        String id = "AC" + String.format("%04d", nextId.getAndIncrement());
        Account account = new Account(id, ownerName, openingBalance);
        repository.save(account);
        return account;
    }

    public Account getAccount(String accountNumber) throws AccountNotFoundException {
        Account account = repository.findById(accountNumber);
        if (account == null)
            throw new AccountNotFoundException(accountNumber);
        return account;
    }

    public void deposit(String accountNumber, long amount) throws BankException {
        if (amount <= 0)
            throw new InvalidAmountException(amount);
        getAccount(accountNumber).deposit(amount);
    }

    public void withdraw(String accountNumber, long amount) throws BankException {
        if (amount <= 0)
            throw new InvalidAmountException(amount);
        Account account = getAccount(accountNumber);
        if (account.getBalance() < amount)
            throw new InsufficientFundsException(amount - account.getBalance());
        account.withdraw(amount);
    }

    public void transfer(String from, String to, long amount) throws BankException {
        if (amount <= 0)
            throw new InvalidAmountException(amount);
        Account source = getAccount(from);
        Account destination = getAccount(to);
```

Code Screenshot 5

Id.java 1

14-B > Id.java > ...

```java
package model.annotation;

import java.lang.annotation.*;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
public @interface Id {
}

```

Code Screenshot 6

Id.java 1 InsufficientFundsException.java 1

14-B > InsufficientFundsException.java > ...

```java
package exception;

public class InsufficientFundsException extends BankException {
    private final long shortfall;

    public InsufficientFundsException(long shortfall) {
        super("Insufficient funds. Shortfall: " + shortfall);
        this.shortfall = shortfall;
    }

    public long getShortfall() {
        return shortfall;
    }
}

```

Code Screenshot 7

InvalidAmountException.java 1

14-B > InvalidAmountException.java > ...

```java
package exception;

public class InvalidAmountException extends BankException {
    public InvalidAmountException(long amount) {
        super("Invalid amount: " + amount);
    }
}

```

Code Screenshot 8

MiniBank.java 1

14-B > MiniBank.java > MiniBank > main(String[])

```java
package app;

import model.Account;
import repository.Repository;
import service.*;
import exception.BankException;
import java.nio.file.Path;
import java.util.*;
import java.util.concurrent.ExecutionException;

public class MiniBank {
    Run | Debug
    public static void main(String[] args) {
        Repository<Account> repository = new Repository<>();
        BankService bank = new BankService(repository);
        AccountFileStore fileStore = new AccountFileStore(Path.of("data", "accounts.txt"));
        ReportService reports = new ReportService();

        try {
            fileStore.loadInto(bank);
        } catch (Exception e) {
            System.out.println("Could not load saved accounts: " + e.getMessage());
        }

        try (Scanner sc = new Scanner(System.in)) {
            while (true) {
                System.out.println("\n===== MiniBank =====");
                System.out.println("1. Open account");
                System.out.println("2. Deposit");
                System.out.println("3. Withdraw");
                System.out.println("4. Transfer");
                System.out.println("5. Statement");
                System.out.println("6. List accounts");
                System.out.println("7. Run report");
                System.out.println("8. Process sample batch");
                System.out.println("0. Save and exit");
                System.out.print("Choice: ");
                String choice = sc.nextLine().trim();
                try {
                    switch (choice) {
                        case "1" -> {
                            System.out.print("Owner name: ");
                            String owner = sc.nextLine();
                            System.out.print("Opening balance: ");
                            long amount = Long.parseLong(sc.nextLine());
                            System.out.println("Created: " + bank.openAccount(owner, amount));
                        }
                        case "2" -> {
```

Code Screenshot 9

ReportService.java 1

14-B > ReportService.java > ...

```java
package service;

import model.Account;
import java.util.Comparator;
import java.util.List;

public class ReportService {
    public String buildReport(List<Account> accounts) {
        long total = accounts.stream().mapToLong(Account::getBalance).sum();
        StringBuilder report = new StringBuilder("=== MiniBank Report ===\n");
        accounts.stream().sorted(Comparator.comparingLong(Account::getBalance).reversed())
                .forEach(a -> report.append(a).append('\n'));
        report.append("Total accounts: ").append(accounts.size()).append('\n');
        report.append("Total balance: ").append(total);
        return report.toString();
    }
}

```

Code Screenshot 10

Repository.java 1

14-B > Repository.java > Repository<T> > saveAll(T...)

```java
package repository;

import model.annotation.Id;
import java.lang.reflect.Field;
import java.util.ArrayList;
import java.util.concurrent.ConcurrentHashMap;

public class Repository<T> {
    private final ConcurrentHashMap<String, T> store = new ConcurrentHashMap<>();

    public T save(T entity) {
        if (entity == null) throw new IllegalArgumentException("Entity cannot be null");
        store.put(readId(entity), entity);
        return entity;
    }

    @SafeVarargs
    public final void saveAll(T... items) {
        for (T item : items) save(item);
    }

    public T findById(String id) { return store.get(id); }
    public ArrayList<T> findAll() { return new ArrayList<>(store.values()); }
    public boolean delete(String id) { return store.remove(id) != null; }
    public int count() { return store.size(); }

    private String readId(T entity) {
        for (Field field : entity.getClass().getDeclaredFields()) {
            if (field.isAnnotationPresent(Id.class)) {
                try {
                    field.setAccessible(true);
                    Object value = field.get(entity);
                    if (value == null) throw new IllegalStateException("@Id field cannot be null");
                    return value.toString();
                } catch (ReflectiveOperationException e) {
                    throw new IllegalStateException("Unable to read @Id field", e);
                }
            }
        }
        throw new IllegalArgumentException("Entity has no field annotated with @Id");
    }
}

```

Code Screenshot 11

Repository.java 1 TransactionProcessor.java 1

14-B > TransactionProcessor.java > ...

```java
package service;

import java.util.List;
import java.util.concurrent.*;

public class TransactionProcessor implements AutoCloseable {
    private final ExecutorService executor = Executors.newFixedThreadPool(4);

    public void processBatch(List<Runnable> transactions) throws InterruptedException, ExecutionException {
        List<Future<?>> futures = executor.invokeAll(transactions);
        for (Future<?> future : futures)
            future.get();
    }

    @Override
    public void close() {
        executor.shutdown();
    }
}

```

Code Screenshot 12

Validator.java 1

14-B > Validator.java > ...

```java
package util;

import java.util.regex.Pattern;

public final class Validator {
    private static final Pattern MOBILE = Pattern.compile("[6-9][0-9]{9}");

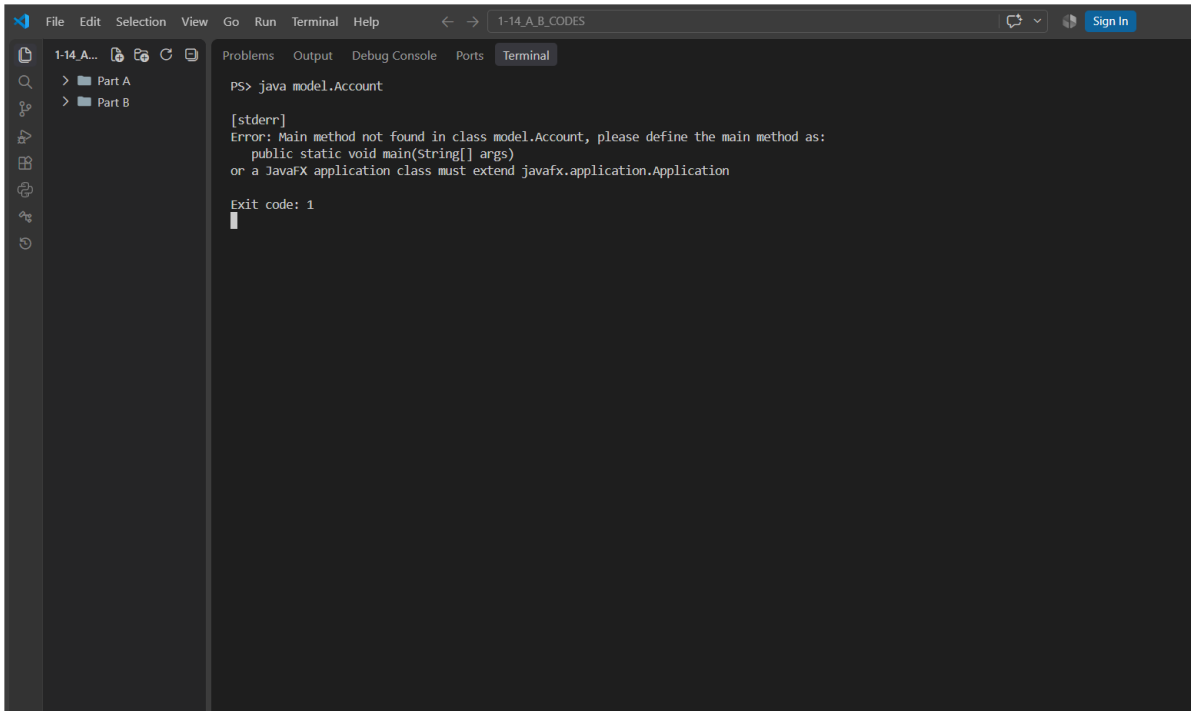
    private Validator() {
    }

    public static boolean isValidMobile(String value) {
        return value != null && MOBILE.matcher(value).matches();
    }
}

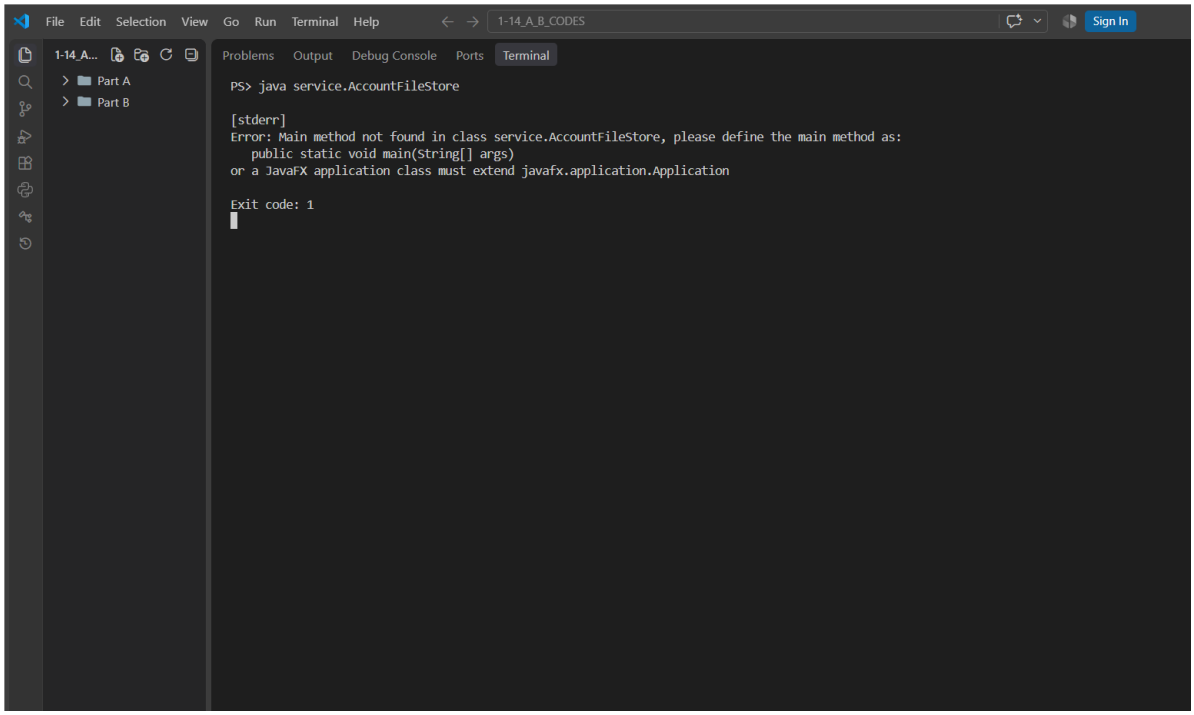
```

Code Screenshot 13

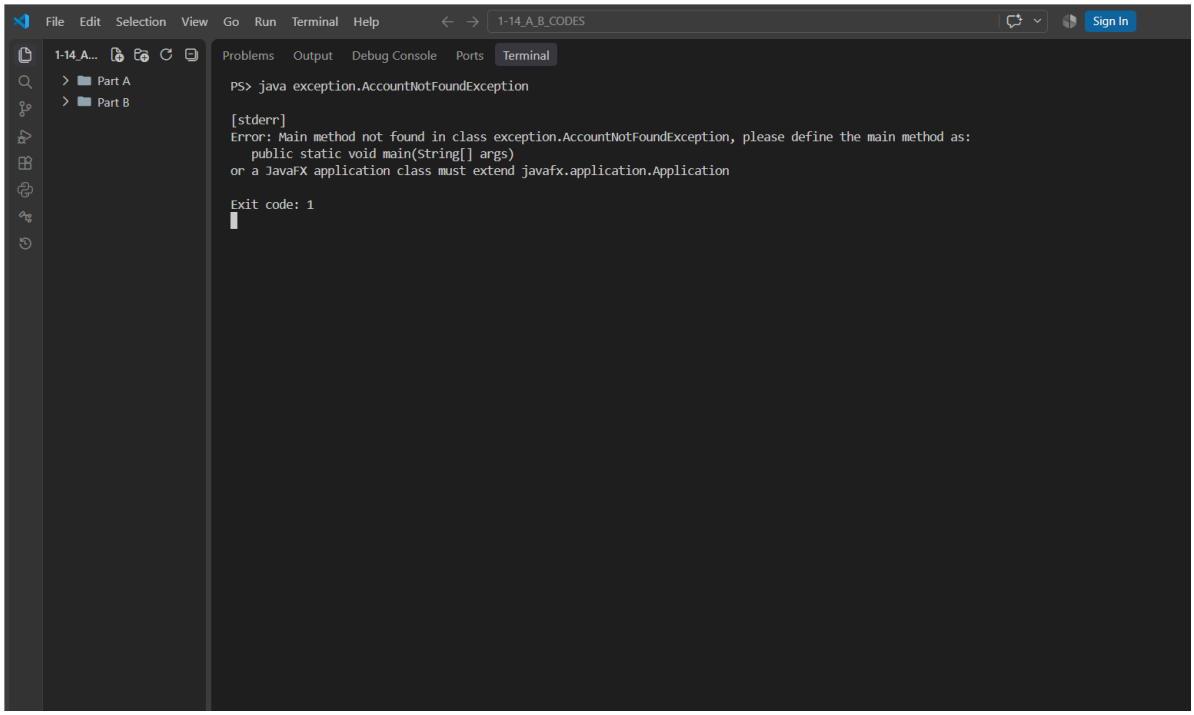
OUTPUT SCREENSHOT:



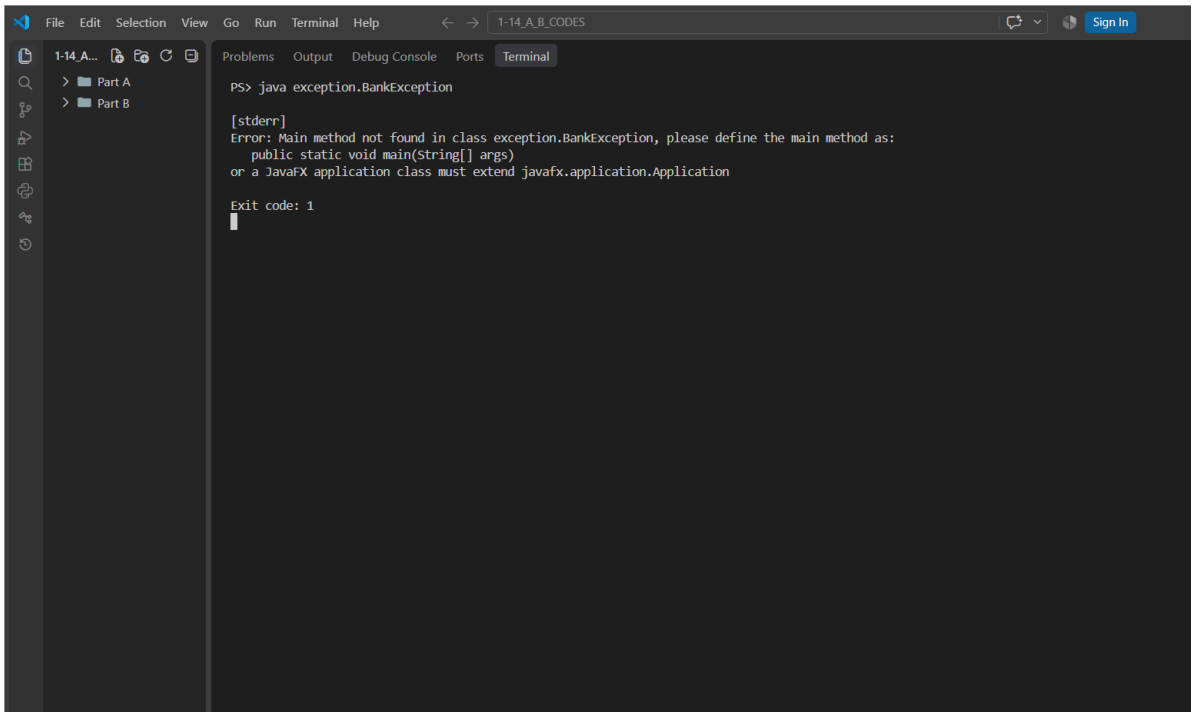
Output Screenshot 1



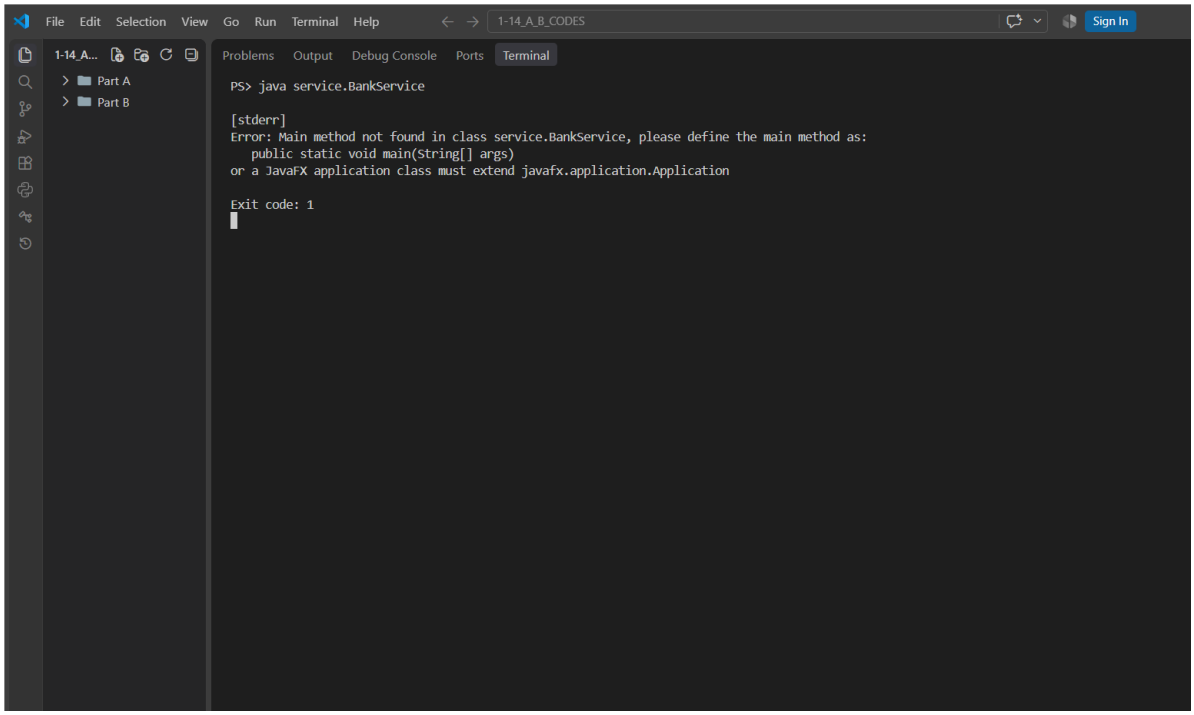
Output Screenshot 2



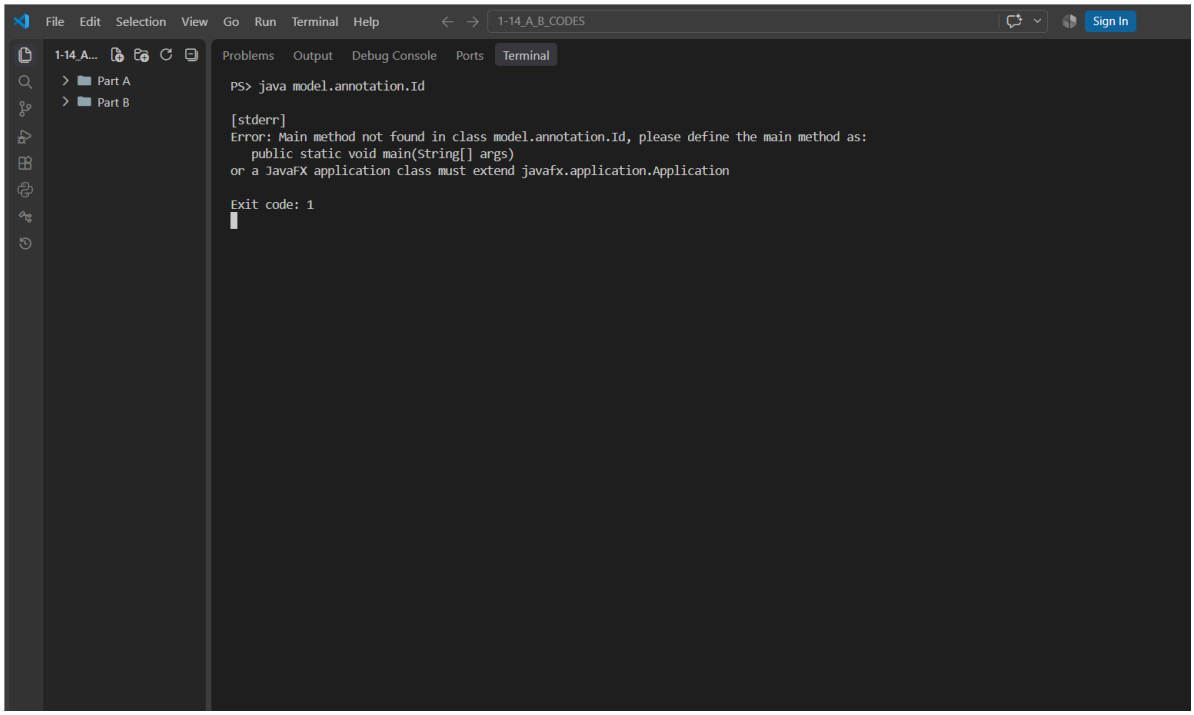
Output Screenshot 3



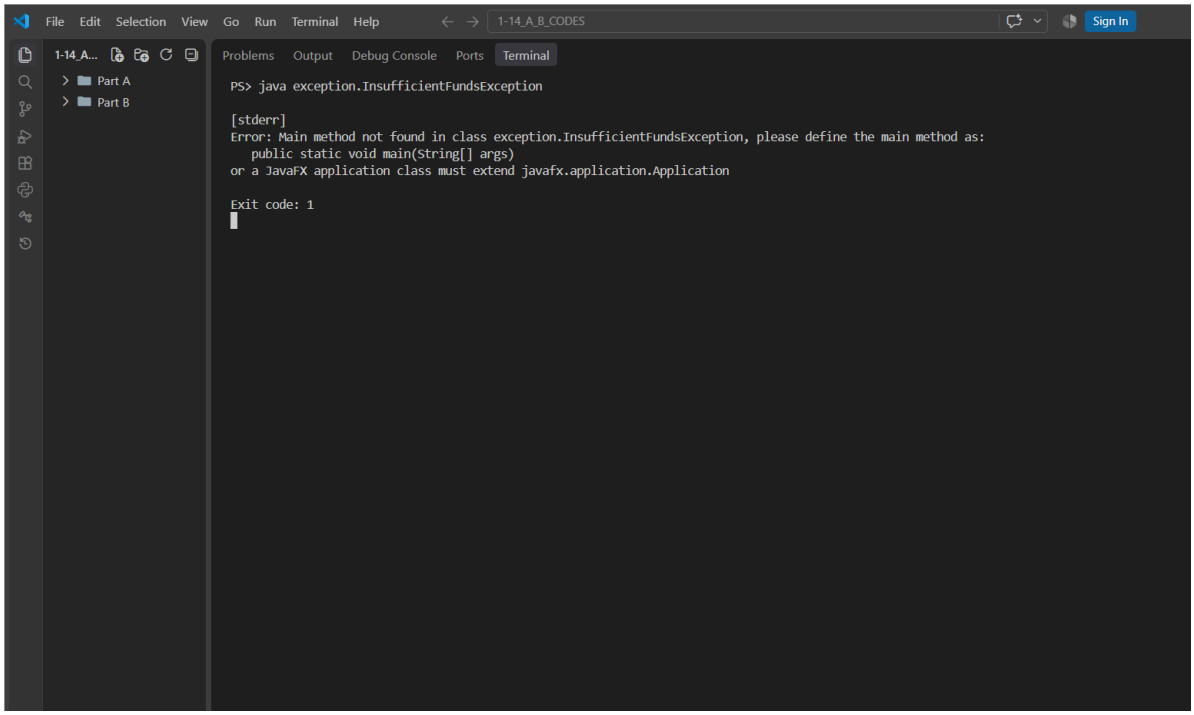
Output Screenshot 4



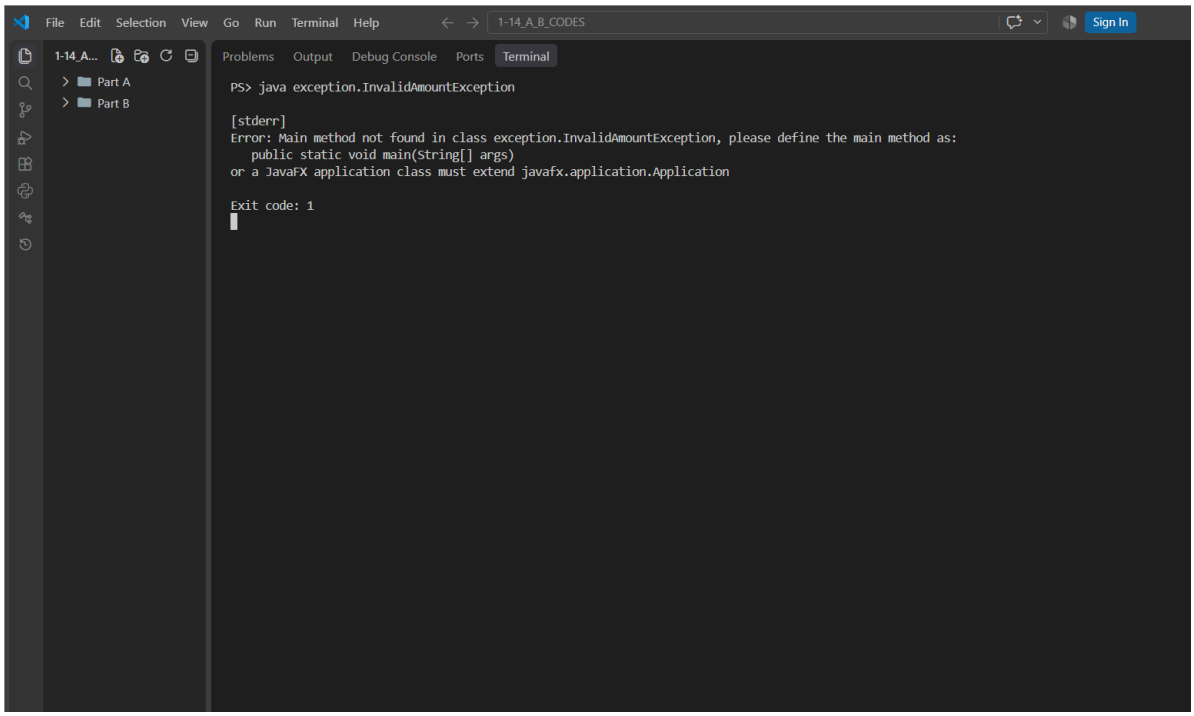
Output Screenshot 5



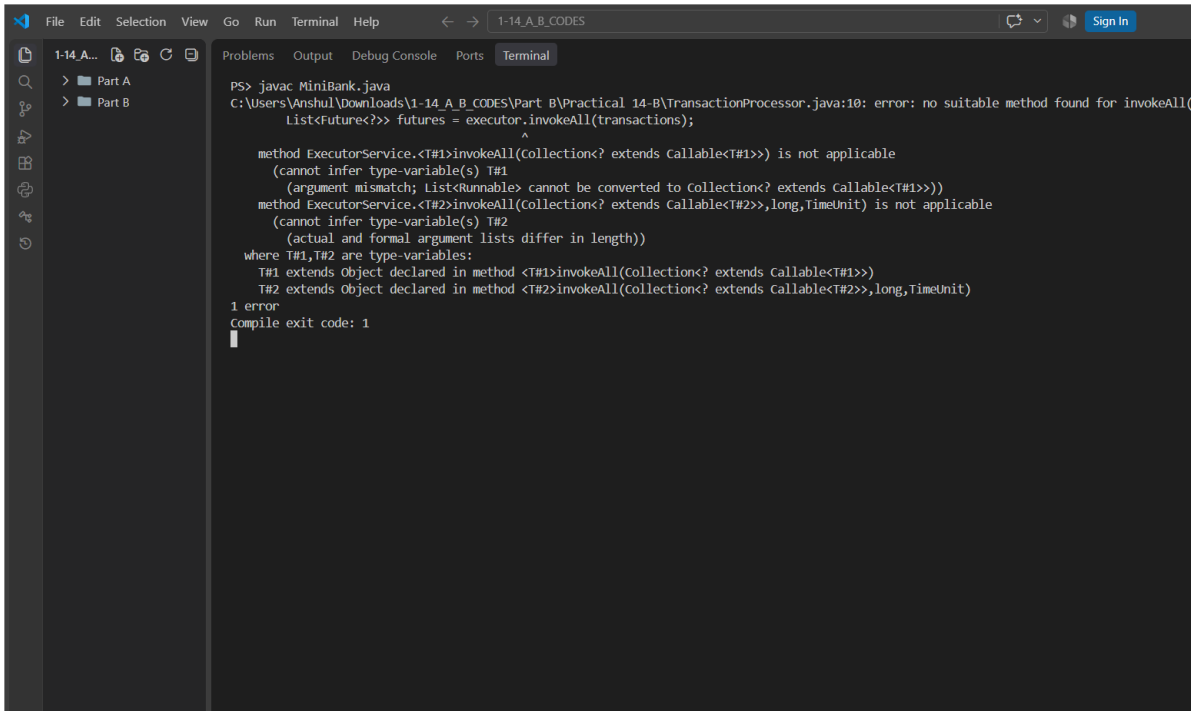
Output Screenshot 6



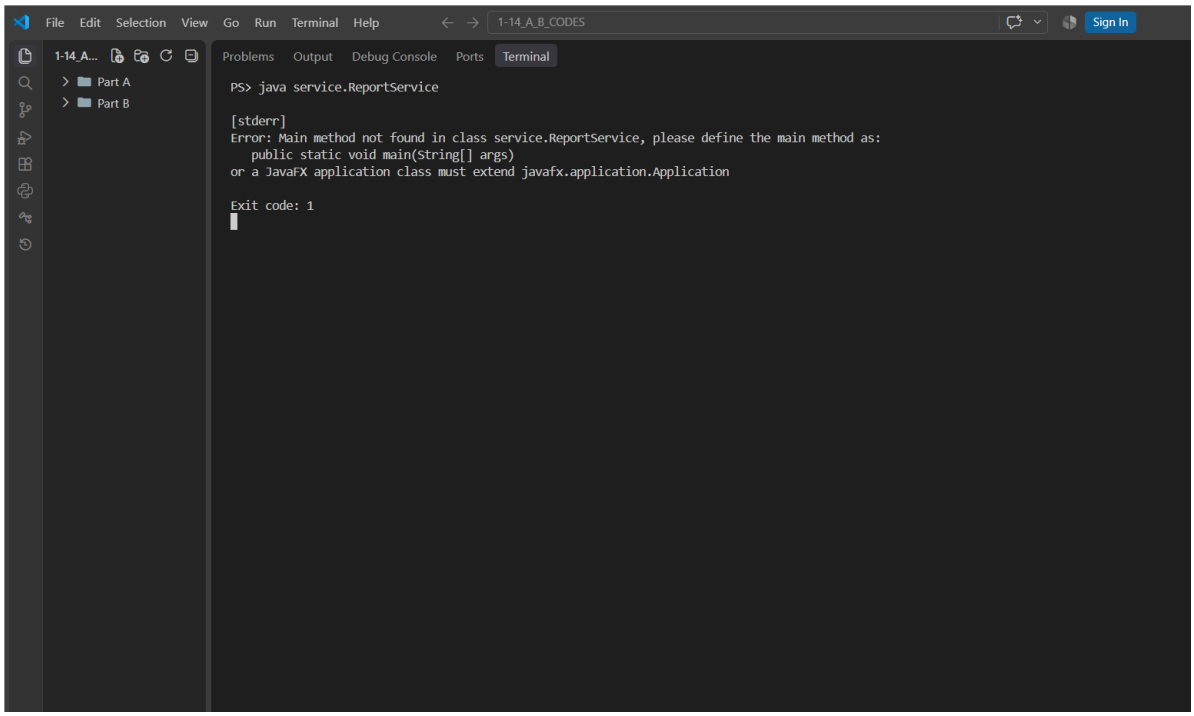
Output Screenshot 7



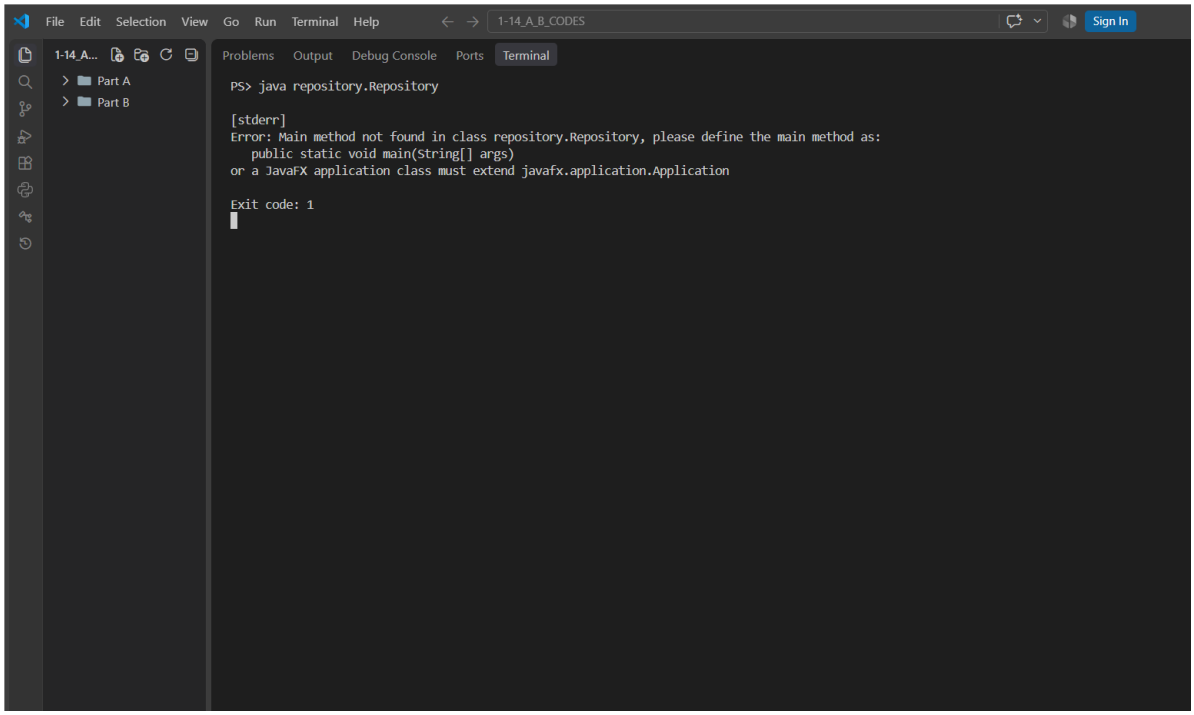
Output Screenshot 8



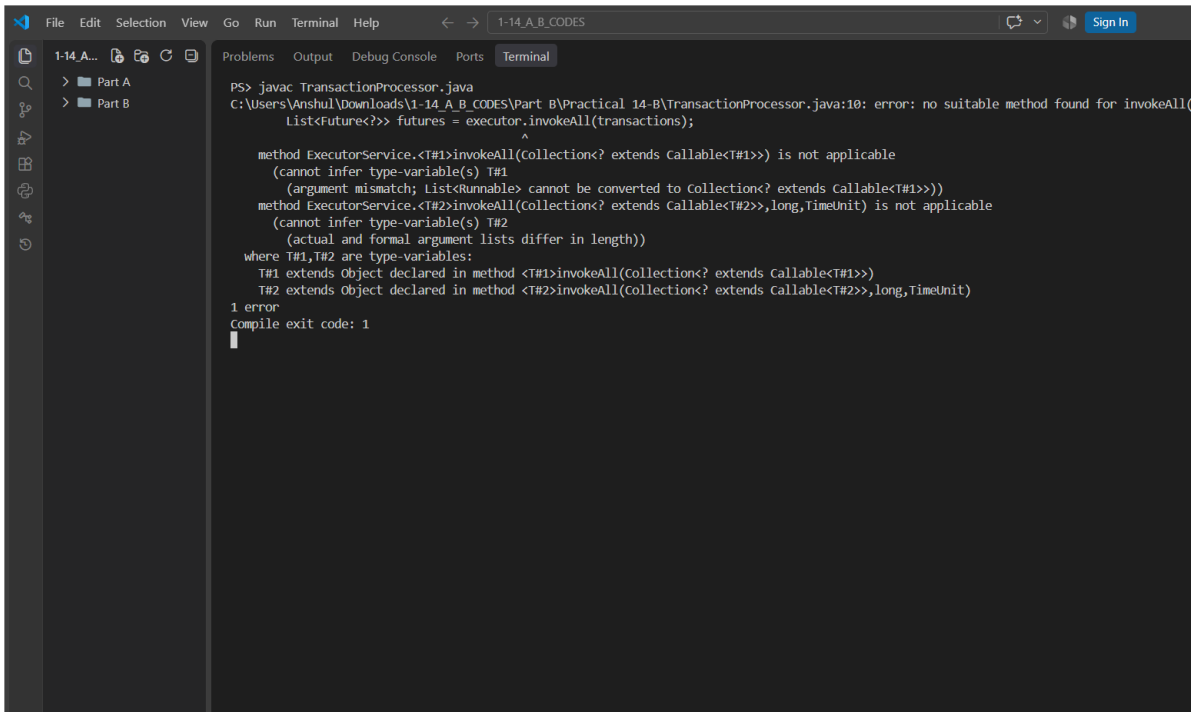
Output Screenshot 9



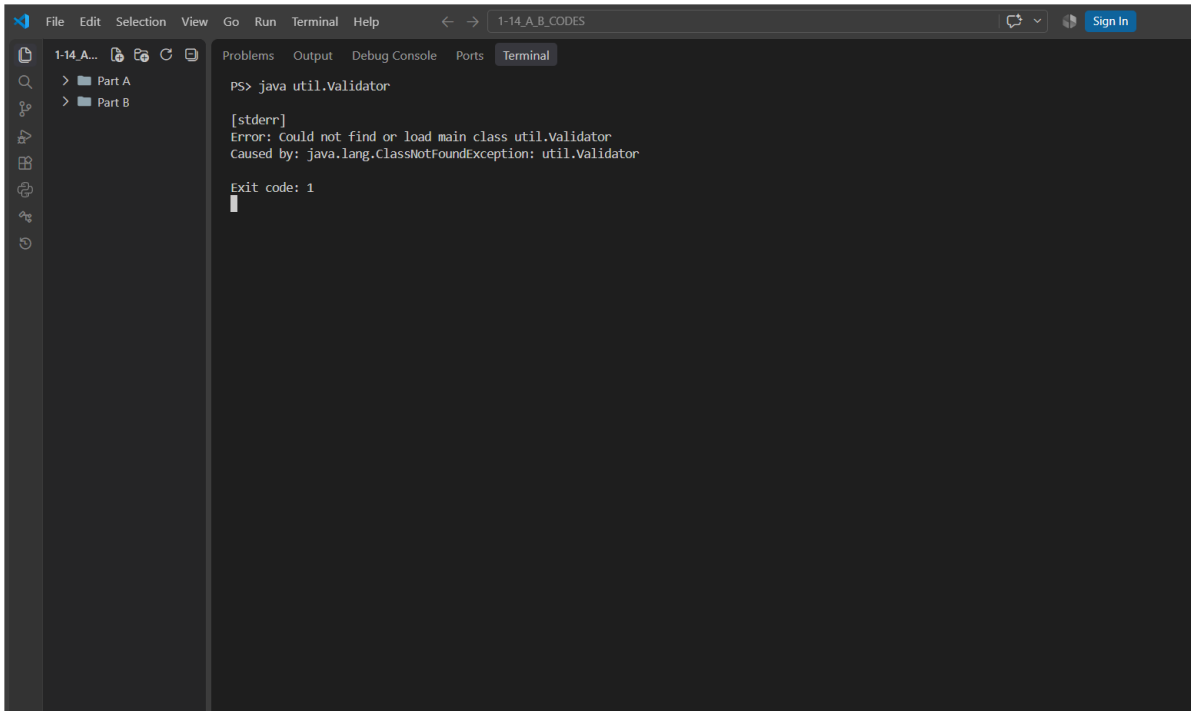
Output Screenshot 10



Output Screenshot 11



Output Screenshot 12



Output Screenshot 13

LEARNING OUTCOMES:

The student can integrate all modules into one tested, packaged, concurrent application — industry-ready. (CO1–CO6)